

Efficient Data Storage Systems

University of Minho

Guide 4

Storage optimizations: Tiering

2026

LevelDB stores data using a Log-Structured Merge Tree (LSM-tree). During normal operation, it creates multiple files such as write-ahead logs (WAL) and SSTables. In production environments, these files are often placed on different storage tiers. In this guide you will implement a simple **tiering mechanism** at the file system level using FUSE. The goal is to control where LevelDB files are stored depending on their type.

1 Setup

Use the VM and hashtable from *Guide0*, the FUSE file system from *Guide2*, and the LevelDB key-value store from *Guide3*. Modify LevelDB's configuration in `myapp.cpp` so that SST files are limited to 2 MB.

```
options.max_file_size = 2 * 1024 * 1024;          // SST file size (.ldb)
```

2 Exercises

Implement a tiering mechanism in your FUSE file system that redirects LevelDB files to different directories (simulating different devices). Intercept all file system operations that use file paths (e.g., `open`, `create`, `rename`, `read`, `write`).

1. Modify the file system so that WAL files (files with extension `.log`) are stored in the directory `/logs`.
Hint: Implement a helper function that rewrites file paths before executing the underlying system call.
2. Extend the previous mechanism so that SST files (extension `.ldb`) are stored in the directory `/sst`. Verify your implementation by inserting data into LevelDB (using the `writer` and workload) and inspecting the directories.
3. The number of SST files may grow quickly. Partition the `sst` directory into two subdirectories (`/sst/odd` and `/sst/even`), and distribute SST files between these directories (**Hint:** A simple solution is to randomly select one of the directories using `random()`). Additionally, maintain a `HashMap` that represents the namespace of the key-value store files, mapping the original path requested by the application with the actual path used in the underlying storage. Your implementation must ensure that the `HashMap` is thread-safe.

3 Extras / Homework

1. Persist the `HashMap` so that the namespace can be recovered when the file system restarts.

2. Redesign the directory layout to simulate heterogeneous storage devices. Create two directories, `fast` and `slow`. Operations targeting the `slow` directory should be artificially delayed (for example, using `sleep`). Modify your tiering policy to maximize the performance of the key-value store.

Hint: Check the paper “Keigo: Co-designing Log-Structured Merge Key-Value Stores with a Non-Volatile, Concurrency-aware Storage Hierarchy” (<https://www.vldb.org/pvldb/vol118/p2872-macedo.pdf>).

Learning outcomes

Implement a tiering system at the file system level for LSM-based key-value stores. Design metadata structures for namespace management.